

SIH26092

Product Requirements Document

AI-Powered Scheme Matching & Channel Partner Routing Platform

SIH 2026 Hackathon | Version 1.0 | Implementation Ready

A focused product and engineering plan for a rapid, credible prototype that connects eligible beneficiaries to suitable government financial schemes and authorized channel partners.

Document	PRD / Implementation Plan
Problem Statement	SIH26092
Primary Users	Marginalized entrepreneurs and students
Core Capabilities	Scheme Recommendation, Financial Calculator, Partner Locator & Router
Prototype Target	Working end-to-end demo by submission deadline

1. Product Summary

1.1 Problem

Eligible Scheduled Caste beneficiaries often struggle to understand which NSFDC financial scheme applies to their situation, whether they satisfy eligibility requirements, how much they can borrow, what their approximate repayment burden will be, which authorized Channel Partner can process their application, where that partner is located, what documents they need, and why a particular scheme or partner has been recommended.

1.2 Product Vision

Build a multilingual, AI-assisted digital platform that converts a beneficiary's natural-language requirements into: eligible scheme → explainable recommendation → financial estimate → suitable nearby channel partner → application guidance.

The platform should make government financial assistance feel like a guided decision process rather than a search through fragmented government documents.

2. Hackathon Problem Statement

SIH26092: MoSJE AI scheme matching for marginalized entrepreneurs.

The solution must demonstrate three central capabilities:

- **Smart Scheme Recommender**
- **Financial Calculator**
- **Geo-Spatial Partner Locator & Router**

3. Product Goals

ID	Goal	Description
G1	Intelligent scheme matching	Use SC status, income, purpose, activity/project, project cost, education status and related
G2	Explainable recommendations	Show why a scheme was recommended instead of presenting only a scheme name.
G3	Financial transparency	Show maximum eligible loan, financing percentage, interest rate, approximate EMI, rep
G4	Channel partner discovery	Identify authorized partners that can potentially process the selected scheme.
G5	Geospatial routing	Rank partners using scheme compatibility, distance and available authoritative perform
G6	Accessibility	Provide a simple, multilingual and mobile-friendly interface.

4. Non-Goals

- Actual loan disbursement or banking transactions.
- Complete integration with every government backend.
- Production-grade authentication and authorization.
- Full loan underwriting, credit scoring or fraud detection.
- Kubernetes or microservices infrastructure for the prototype.
- Native Android/iOS applications.
- Coverage of every possible Indian government scheme.

- Guaranteed loan approval.

The platform provides guidance and intelligent matching, not loan approval.

5. Target Users

Persona 1: Marginalized Entrepreneur

Example: SC beneficiary, family income ₹3.5 lakh/year, wants to start a ₹3 lakh tailoring business. Needs a suitable scheme, loan amount, EMI, nearby authorized partner and required documents.

Persona 2: Student

Example: SC student pursuing B.Tech and seeking education financing. Needs education-loan eligibility, financing information, repayment information and application guidance.

Persona 3: Existing Small Business Owner

Example: Existing micro-business owner seeking additional financing. Needs an appropriate scheme, financing limits, channel partner and application process.

6. Core User Journey

```
Landing Page
↓
Find the right scheme
↓
Basic Assessment
↓
User Information
↓
AI understands requirements
↓
Eligibility Engine
↓
Scheme Ranking
↓
Recommendation
↓
Financial Calculator
↓
Partner Locator
↓
Partner Ranking
↓
Application Guidance
```

7. Core Product Modules

Module 1: Smart Assessment

Inputs: applicant type, SC status, annual family income, purpose, activity/project, project cost, education status, course where applicable, and location.

The UI should use a conversational or guided experience instead of a large government-style form.

```
Example:
"I want to start a ₹3 lakh tailoring business. My family's income is ₹3.5 lakh."

AI extraction:
purpose = business
activity = tailoring
project_cost = 300000
annual_income = 350000
```

Module 2: Scheme Recommendation Engine

```
User Input
↓
AI/NLP Extraction
↓
Structured User Profile
↓
Eligibility Rules
↓
Candidate Schemes
↓
Scheme Scoring
↓
Ranked Recommendations
```

Critical design rule: AI does not determine eligibility. AI understands language. Deterministic rules determine eligibility. The database provides authoritative information.

Scheme Matching Logic

Factor	Initial Weight
Eligibility	40%
Activity Match	25%
Financial Fit	20%
Partner Availability	15%

The score is an internal recommendation score, not a government-approved probability.

Module 3: Financial Calculator

The calculator is scheme-aware. Inputs include loan amount, interest rate, tenure and moratorium. Outputs include estimated EMI, total interest, total repayment and repayment duration.

Financial parameters should come from the database rather than being hardcoded in frontend code.

Module 4: Channel Partner Locator

```
Location + Selected Scheme
↓
Authorized Partners
↓
Scheme compatibility
↓
Geospatial filtering
↓
Partner ranking
```

Partner Ranking

Factor	Initial Weight
Scheme Compatibility	40%
Distance	25%
Performance	20%
Availability	15%

If an official performance or availability metric is unavailable, the system must not fabricate it. It should explicitly show that the metric is unavailable from the current authoritative source.

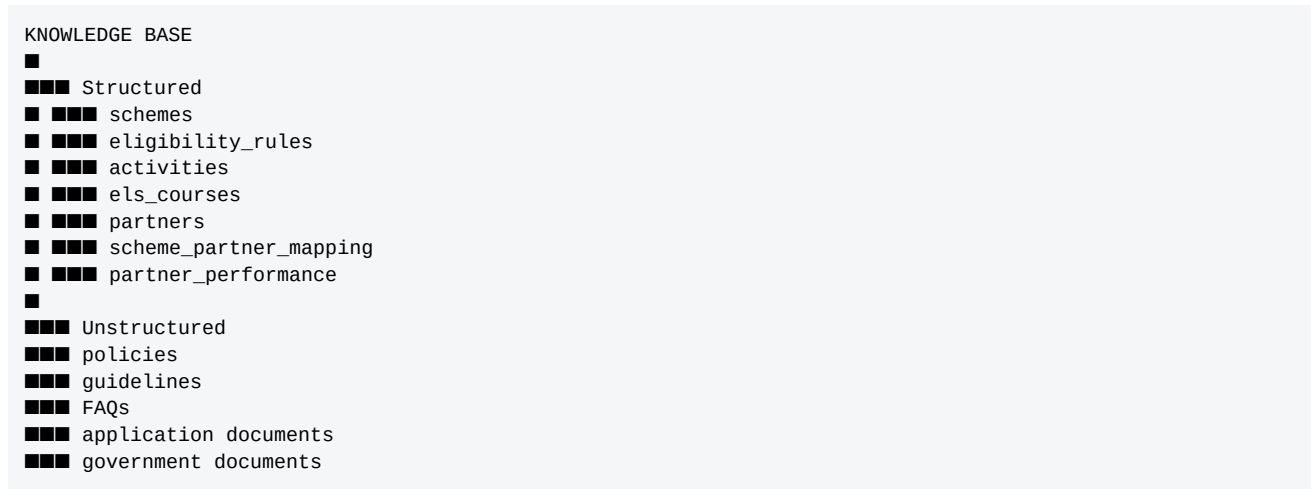
8. Current Scheme Master

The initial knowledge base contains five NSFDC schemes:

- Micro Finance Scheme
- Term Loan
- Aajeevika Micro-Finance Yojana
- Udyam Nidhi Yojana
- Educational Loan Scheme

The current Scheme Master includes scheme type, purpose, project cost limits, maximum loan, financing percentage, interest rates, repayment, moratorium, channel types, target group and application mode. This dataset must be versioned as government information changes.

9. Knowledge Base Architecture



Data Governance

Every important government fact should ultimately have source_id, source_url, authority, published date, effective date, retrieval timestamp and version.

Never fabricate government eligibility, interest rates, partner status, NPA data, or approval status. If authoritative data is unavailable, state that it is unavailable.

10. Database Design

Technology: PostgreSQL. Potential future extension: pgvector for semantic retrieval.

Core Tables

Table	Purpose
schemes	Scheme financial and product parameters
eligibility_rules	Structured eligibility conditions
activities	Government-listed activity taxonomy
els_courses	Educational Loan Scheme course categories
partners	Authorized channel partner master
scheme_partner_mapping	Scheme-to-partner authorization mapping

Table	Purpose
partner_performance	Official partner performance/availability metrics where available
application_requirements	Scheme-specific document requirements
faqs	Question-answer knowledge for guided support
sources	Government source and version registry

10.1 Schemes

```
schemes
-----
id
scheme_id
name
scheme_type
purpose
project_cost_min
project_cost_max
max_loan_amount
financing_percentage
nsfdc_interest_rate
beneficiary_interest_rate
repayment_period
moratorium_period
installment_frequency
target_group
application_mode
status
effective_from
effective_until
source_id
created_at
updated_at
```

10.2 Eligibility Rules

```
eligibility_rules
-----
id
scheme_id
field
operator
value
unit
priority
explanation
source_id
effective_from
effective_until
```

Eligibility conditions should be stored as data rather than embedding every condition directly in Python.

10.3 Activities

```
activities
-----
id
name
sector
sub_sector
keywords
aliases
```

10.4 Partners

```
partners
-----
id
```

```
partner_id
name
partner_type
state
district
address
latitude
longitude
phone
email
website
status
source_id
```

10.5 Scheme-Partner Mapping

```

scheme_partner_mapping
-----
id
scheme_id
partner_id
authorization_status
geographic_scope
source_id

```

10.6 Partner Performance

```
partner_performance
-----
id
partner_id
period
sanctioned_amount
disbursed_amount
utilization_percentage
beneficiary_count
pending_amount
npa_percentage
overdue_amount
status
as_of_date
source_id
```

10.7 Application Requirements

```
application_requirements
-----
id
scheme_id
document_name
mandatory
description
source_id
```

11. System Architecture

[illegible]

[illegible]

Architecture decision: Use a modular monolith. Do not build microservices for the hackathon. One backend, one database and one frontend are faster to implement, debug and deploy.

12. Technology Stack

Layer	Technology	Reason
Frontend	Next.js + TypeScript	Fast development, strong ecosystem, good deployment
UI	Tailwind CSS + shadcn/ui	Rapid, consistent interface development
Maps	Leaflet + OpenStreetMap	Simple geospatial visualization
Backend	FastAPI + Python	Fast API development and natural fit for rules/AI/data
API	REST + JSON	Simple frontend/backend integration
Database	PostgreSQL	Reliable structured data and extensibility
ORM	SQLAlchemy	Safe and maintainable DB access
Validation	Pydantic	Strong API request/response validation
AI	LLM API	Intent and entity extraction, multilingual understanding
Vector Search	pgvector, later	RAG over official documents
Deployment	Vercel + Render/Railway + Supabase	Fast prototype deployment
Version Control	GitHub	Team collaboration and deployment integration

13. Backend Architecture

```
backend/
  app/
  main.py
  api/
    schemes.py
    recommendations.py
    calculator.py
    partners.py
    chat.py
  models/
    scheme.py
    partner.py
    activity.py
  schemas/
    recommendation.py
    calculator.py
  services/
```

```
■ ■■■ recommendation_engine.py
■ ■■■ eligibility_engine.py
■ ■■■ partner_router.py
■ ■■■ financial_calculator.py
■ ■■■ ai_service.py
■■■ db/
■■■ database.py
■■■ seed.py
```

14. API Design

Method	Endpoint	Purpose
GET	/api/schemes	List available schemes
GET	/api/schemes/{scheme_id}	Get scheme details
POST	/api/recommend	Generate ranked scheme recommendations
POST	/api/calculate-emi	Calculate scheme-aware EMI
GET	/api/partners/nearby	Find nearby compatible partners
POST	/api/chat	Natural-language AI assistant

Recommendation Request

```
{
  "is_sc": true,
  "annual_income": 350000,
  "purpose": "business",
  "activity": "tailoring",
  "project_cost": 300000,
  "location": {
    "lat": 16.5,
    "lng": 80.6
  }
}
```

Recommendation Response

```
{
  "recommendations": [
    {
      "scheme_id": "NSFDC-TL",
      "scheme_name": "Term Loan",
      "score": 94,
      "reasons": [
        "Income is within the eligibility limit",
        "Project cost fits the scheme",
        "Tailoring is an eligible activity"
      ]
    }
  ]
}
```

15. Frontend & UX

Frontend: Next.js, TypeScript, Tailwind CSS and shadcn/ui.

Pages

```
/
■■■ Landing
■■■ /assess
■ ■■■ Assessment
■■■ /results
```

```
■ ■■■ Recommendations
■■■ /schemes
■ ■■■ Scheme Explorer
■■■ /partners
■ ■■■ Partner Map
■■■ /chat
■■■ AI Assistant
```

Primary UX Principles

- **No government-form experience:** ask one focused question at a time.
- **Explain everything:** show the reasons behind recommendations.
- **Progressive disclosure:** show the key answer first and details on demand.
- **Mobile-first:** the core assessment and results must work well on small screens.
- **Clear uncertainty:** distinguish official facts from estimates and unavailable data.

Primary Screens

Landing: “Find the Government Support That's Right for You” with Start a Business, Education Loan and Explore Schemes actions.

Assessment: SC eligibility, annual income, purpose, activity/course, project cost and location.

Recommendation: best match, score, financing information and reasons.

Partner: map, nearby partners, distance, scheme compatibility and directions.

16. Multilingual Strategy

Initial UI languages: English, Hindi and Telugu, provided translations can be validated by the team. The architecture should support additional languages later.

```
Telugu / Hindi / English
↓
AI interpretation
↓
English structured representation
↓
Decision engine
↓
Localized response
```

17. AI + RAG Strategy

The chatbot is a supporting interface, not the product itself.

```
Natural-language request
↓
LLM intent/entity extraction
↓
Structured user profile
↓
Deterministic recommendation engine
↓
Ranked scheme
↓
RAG for supporting policy/document explanation
```

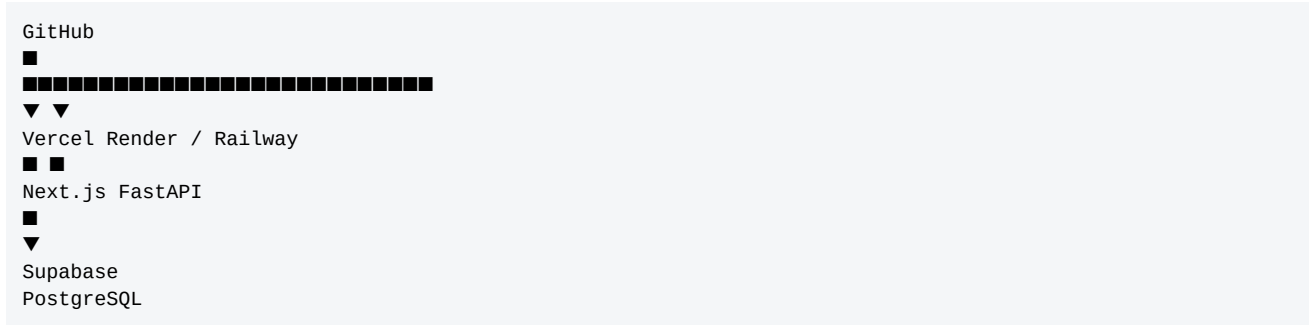
For the first prototype, RAG should remain lightweight. The rule engine is the primary source of eligibility decisions. RAG is mainly for explaining policies, documents, FAQs and official guidance.

18. Data Storage

Data	Storage
Schemes, rules, activities, partners	PostgreSQL
Government document metadata	PostgreSQL
Original PDFs/forms/guidelines	Object storage such as Supabase Storage
Document embeddings	pgvector when RAG is enabled
Source/version registry	PostgreSQL

Do not store large PDF binaries directly in PostgreSQL. Store original documents in object storage and retain metadata, source URLs and extracted knowledge in the database.

19. Deployment



Environment variables should hold database URLs, AI keys, CORS origins and other secrets. Never commit secrets to GitHub.

20. Team Division

Role	Primary Responsibilities	Key Deliverables
Member 1: Backend	FastAPI, DB integration, eligibility, recommendation engine, calculator, AI, contracts, /schemes	Backend API, EMI calculator, AI contracts, /schemes
Member 2: Data/KB	Scheme, rules, activities, ELS, partners, mappings, recommendations, knowledge base	Scheme database, rules, activities, ELS, partners, mappings, recommendations, knowledge base
Member 3: Frontend/UX	Next.js, assessment, results, calculator, partner UI, results, assessment, Results, Calculator, Map	Frontend UI, assessment, results, calculator, partner UI, results, assessment, Results, Calculator, Map
Member 4: AI/Integration	LLM extraction, chat, RAG, integration, deployment, AI type + deployment + final integration	AI type + deployment + final integration

If a fifth member is available, dedicate them to Partner Master, scheme-partner mapping and geospatial routing.

21. Priority Matrix

Priority	Features
P0: Must Work	Scheme database, eligibility engine, recommendation engine, EMI calculator, partner database
P1: Differentiators	Natural-language AI, explainable recommendations, multilingual UI, document checklist
P2: Nice-to-Have	RAG, admin dashboard, analytics, advanced performance ranking, user accounts

22. Rapid Implementation Schedule

Time	Phase	Work
0–1 hr	Foundation	GitHub, repo structure, PostgreSQL schema, environment, FastAPI, Next.js
1–2 hr	Data	Import Scheme Master, activities, eligibility and partner data
2–4 hr	Backend	Scheme API, recommendation API, rules, calculator, partner API
4–6 hr	Frontend	Landing, assessment, results, calculator, partner map
6–7 hr	AI	Natural-language input, entity extraction, recommendation integration, chat
7–8 hr	Polish	Responsive UI, loading/error states, multilingual support, explanations
8–9 hr	Testing	End-to-end demo scenarios and deployment verification

23. Hero Demo Scenario

Recommended primary demo: **an SC beneficiary wants to start a ₹3 lakh tailoring business and has a family income of ₹3.5 lakh.**

User:
 "I want to start a ₹3 lakh tailoring business.
 My family income is ₹3.5 lakh."

System:
 1. Understands the request
 2. Identifies business activity
 3. Checks eligibility
 4. Recommends the best-fit scheme
 5. Explains why
 6. Calculates financing / EMI
 7. Finds nearby authorized partners
 8. Shows what to do next

A second demo should cover an education case such as an SC student seeking a B.Tech education loan.

24. Testing Strategy

Unit Tests

- Income and eligibility rules
- Project-cost and loan-limit rules
- Interest and EMI calculations
- Activity matching
- Scheme ranking

API Tests

- POST /recommend
- POST /calculate-emi
- GET /partners/nearby
- GET /schemes

End-to-End Tests

- Frontend → API → database → recommendation → frontend
- Entrepreneur scenario

- Education scenario
- Ineligible applicant scenario
- Partner routing scenario

25. Competitive Differentiation

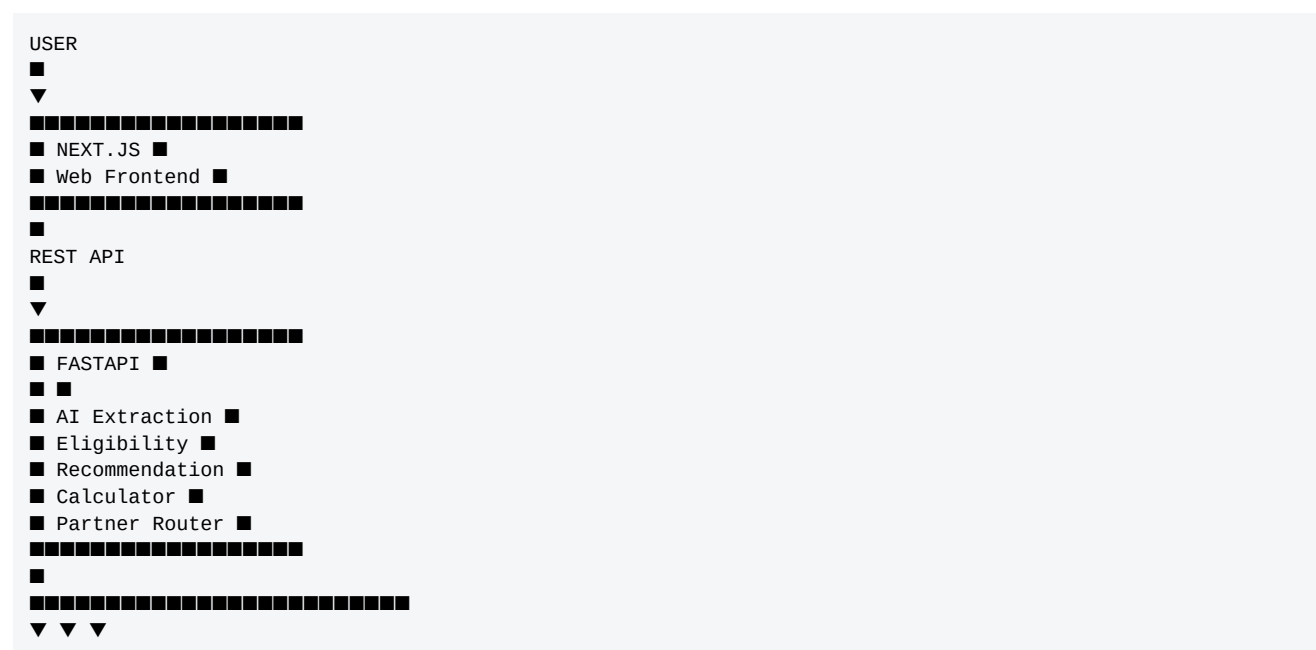
Differentiator	What We Demonstrate
Explainable AI	Not only “what”, but “why” a scheme was recommended
AI + Rules	AI interprets natural language; deterministic rules handle eligibility
Scheme + Partner Matching	Continue beyond scheme discovery into partner routing
Financial Transparency	Immediately show potential financing and repayment burden
Government-Source Grounding	Recommendations and guidance trace back to authoritative information

Recommended positioning: “An explainable decision and routing engine connecting marginalized beneficiaries to the right government financial scheme and the right channel partner.”

26. Definition of Done

- A judge can open the application and start an assessment.
- A user can provide SC status, income, purpose, activity/course, project cost and location.
- The system returns a ranked scheme recommendation with reasons.
- The system calculates an estimated EMI using scheme parameters.
- The system shows nearby authorized partners compatible with the selected scheme.
- The user can understand why the scheme and partner were recommended.
- The core flow works on a mobile-sized screen.
- The deployed prototype works without manual developer intervention.

27. Final Architecture Decision



PostgreSQL pgvector Maps



Government KB



Sources / Policies /

Rules / Partners

Core engineering principle: *AI interprets. Rules decide. Data proves.*

Do not build the project around the chatbot. Build it around the decision engine. The chatbot is simply one interface into the system.

Core product pipeline: Government Knowledge Base → Eligibility Engine → Scheme Recommender → Financial Engine → Partner Router → Explainable Result.